BusinessRegistry.sol

BusinessRegistry.sol

Purpose:

This contract serves as the canonical record of Sotility-approved businesses:

Stores KYC, metadata, public docs via IPFS

Enables DAO or oracles to update status or fields

Maps business roles, payout addresses, category tags, etc.

Notes:

Uses REGISTRAR_ROLE for DAO or oracle-controlled updates

Perfect complement to SotilityExchange.sol

Stores core fields + public IPFS data

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/access/AccessControl.sol";

/**
 * @title BusinessRegistry
 * @dev Stores metadata for verified businesses on the Sotility Exchange.
 */
contract BusinessRegistry is AccessControl {
    bytes32 public constant REGISTRAR_ROLE = keccak256("REGISTRAR_ROLE");

    struct BusinessInfo {
        string name;
        string category;
        string jurisdiction;
        string website;
        string ipfsMetadata;   // Business docs, audits, proof links
        address payoutAddress;
        bool isActive;
    }

    mapping(address => BusinessInfo) public registry;

    event BusinessAdded(address indexed wallet, string name);
    event MetadataUpdated(address indexed wallet, string ipfsHash);
    event BusinessStatusChanged(address indexed wallet, bool active);

    constructor() {
        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
    }

    function registerBusiness(
        address wallet,
        string calldata name,
        string calldata category,
        string calldata jurisdiction,
        string calldata website,
        string calldata ipfsMetadata,
        address payoutAddress
    ) external onlyRole(REGISTRAR_ROLE) {
        require(registry[wallet].payoutAddress == address(0), "Business already exists");

        registry[wallet] = BusinessInfo({
            name: name,
            category: category,
            jurisdiction: jurisdiction,
            website: website,
            ipfsMetadata: ipfsMetadata,
            payoutAddress: payoutAddress,
            isActive: true
        });

        emit BusinessAdded(wallet, name);
    }

    function updateMetadata(address wallet, string calldata newIpfsHash)
        external onlyRole(REGISTRAR_ROLE)
    {
        require(registry[wallet].payoutAddress != address(0), "Business not found");
        registry[wallet].ipfsMetadata = newIpfsHash;
        emit MetadataUpdated(wallet, newIpfsHash);
    }

    function setBusinessActiveStatus(address wallet, bool active)
        external onlyRole(REGISTRAR_ROLE)
    {
        require(registry[wallet].payoutAddress != address(0), "Business not found");
        registry[wallet].isActive = active;
        emit BusinessStatusChanged(wallet, active);
    }

    function getBusiness(address wallet) external view returns (BusinessInfo memory) {
        return registry[wallet];
    }
}
```

Notein
Scan the QR code to download Notein